AI Full-Stack Developer Roadmap with Python (2026–2027)

For a CSE graduate · 3 hours a day · 12 months · Python, FastAPI, React + TypeScript, ML and LLMs

1. How to use this roadmap

Goal. In 12 months you can design, build, deploy and maintain a full-stack web app whose core features run on LLMs and classic ML. Realistic job targets: **AI Application Developer**, **Python Backend Developer**, and **AI Full-Stack Developer** (the combination). **Junior ML Engineer** is possible but needs extra depth in Phases 5–7 (see Career Preparation).

Honest expectations. Twelve months at 3 h/day is about 1,000 hours. That is enough to be employable for application-level AI roles if you build and ship real projects. It is not enough to become a research-level ML expert, and no roadmap should promise that.

Tags used everywhere

- **[MUST]** needed for jobs; do not skip
- **[SHOULD]** high value; learn after the musts
- **[OPTIONAL]** nice to have
- **[ADVANCED]** defer until you have shipped projects and actually need it

Study rules

1. One main phase at a time. The only allowed overlap is a 30-minute daily DSA habit and your current project.
2. Daily 70/20/10 split of 3 hours: about 125 min coding, 35 min theory, 20 min revision.
3. Anti tutorial-hell rule: learn a topic once, close the tutorial, then build from a blank file. Rebuild anything you copied from memory the next day.
4. Every phase ends with something pushed to GitHub with a README. No project, no moving on.
5. Revision: review notes at +1 day, +1 week and +1 month (Anki or plain notes).
6. Use AI assistants to explain errors and review your code, but write the first version yourself until about Month 6. Otherwise you never build the skill that interviews test.

How your current skills are reused. HTML, CSS, JavaScript, React, Tailwind and Git are a real head start: you skip frontend basics and go straight to TypeScript, data fetching and AI UI patterns (streaming, chat, uploads). Your Firebase experience means you already understand hosted auth and databases; you will now build those layers yourself with FastAPI and PostgreSQL, which is what most companies expect.

One deliberate change to the order. The phases keep your numbering, but the timeline interleaves them: you call an LLM API and build a chat UI in Month 5, before deep learning. Application-level LLM work does not need neural-network theory, and shipping early keeps you motivated and employable sooner. Phase 7 (deep learning) is intentionally lean.

Resource note. Links go to official documentation or long-running free courses. Tools in the LLM space (LangChain, vector databases, model names) change fast, so always open the current docs and use the newest stable version instead of copying versions from old tutorials.

2. Phase 1: Python Programming Fundamentals (Month 1)

Why: Python is the language of the AI ecosystem and of FastAPI backends. Everything later assumes you can write clean Python without constant lookups.

Learn

- [MUST] Syntax, variables, data types, operators, conditions, loops
- [MUST] Functions (defaults, `*args`, `**kwargs`, scope), modules and packages
- [MUST] Lists, tuples, dictionaries, sets: when to use each, slicing, iteration
- [MUST] OOP: classes, inheritance vs composition, dunder methods, dataclasses
- [MUST] Exceptions (try/except/else/finally, custom exceptions) and file handling (pathlib, csv, json)
- [MUST] Virtual environments, pip (or uv), pyproject.toml, project layout (src/, tests/, README, .gitignore)
- [MUST] Git/GitHub workflow: branch, commit, pull request, merge, resolving conflicts, never committing secrets
- [SHOULD] Formatting and linting with Ruff, reading tracebacks, using the debugger

Practice (after each topic)

- 5 Exercism exercises per topic, then one mini-program: number guesser (loops), contact book (dicts + JSON file), bank account (OOP), log analyzer (files + exceptions)

- Each mini-program gets its own folder, a README and at least 3 meaningful commits

Resources

- Python Tutorial (official): https://docs.python.org/3/tutorial/
- CS50's Introduction to Programming with Python (free): https://cs50.harvard.edu/python/
- Automate the Boring Stuff with Python (free book): https://automatetheboringstuff.com/
- Exercism Python track (free, interactive): https://exercism.org/tracks/python
- Pro Git (free book): https://git-scm.com/book
- uv documentation: https://docs.astral.sh/uv/
- Practical Python (GitHub course): https://github.com/dabeaz-course/practical-python
- YouTube: freeCodeCamp https://www.youtube.com/@freecodecamp and Corey Schafer https://www.youtube.com/@coreyms (some uploads are older, but the fundamentals have not changed)

Build: Project 1, DeskAutomator (see Projects).

Exit check: you can write a 150-line CLI program from scratch, with a virtual environment, a README and error handling, in under 2 hours.

3. Phase 2: Advanced Python and Computer Science (Month 2)

Why: FastAPI is built on type hints and async; LLM streaming uses generators and async; interviews test data structures and algorithms. Skipping this phase makes every later phase harder.

Learn

- [MUST] Comprehensions, generators and iterators, decorators, context managers (`with`, `contextlib`)
- [MUST] Type hints, `TypedDict` and `Protocol` basics, and a type checker (mypy or pyright)
- [MUST] Asynchronous programming: `async` / `await`, the event loop, `asyncio.gather`, `httpx.AsyncClient`, and when async does NOT help (CPU-bound work)
- [MUST] Data structures: arrays, strings, hash maps, sets, stacks, queues, linked lists, trees, basic graphs, heaps

- [MUST] Algorithms: sorting, binary search, recursion, two pointers, sliding window, BFS/DFS
- [MUST] Big-O time and space complexity: state it for every solution you write
- [SHOULD] Dynamic programming basics, the logging module, Enum, threads vs processes
- [ADVANCED] Metaclasses, C extensions, segment trees, advanced graph algorithms
- Math preview: treat vectors and matrices as lists and later NumPy arrays (full treatment in Phase 5)

Practice

- 30–40 min/day of problems: the first 50 of NeetCode 150 (Easy, then Medium). Solve on your own before reading any solution and write the complexity in a comment
- Re-implement from scratch: a `@timer` decorator, a `@retry` decorator, a generator that reads a large file lazily, a context manager that times a block
- Write an async script that fetches 50 URLs concurrently with a limit, then compare it to the sequential version

Resources

- asyncio docs: https://docs.python.org/3/library/asyncio.html
- typing docs: https://docs.python.org/3/library/typing.html and mypy: https://mypy.readthedocs.io/
- Real Python (free articles): https://realpython.com/
- NeetCode roadmap: https://neetcode.io/roadmap and LeetCode: https://leetcode.com/
- VisuAlgo (visual algorithms): https://visualgo.net/
- Books: *Fluent Python*, 2nd ed. (Ramalho): read the chapters on the data model, generators, decorators and async. *Grokking Algorithms* (Bhargava) for gentle intuition

Build: a small async tool (for example a concurrent API health checker with retries and a summary report), fully typed and tested with pytest.

Exit check: you solve most Easy and some Medium problems in 30 minutes, and can explain generators, decorators and async to a friend.

4. Phase 3: Backend Development with Python (Months 3–4)

Why: The backend is where AI features actually live: it holds your API keys, enforces auth, talks to databases and vector stores, and streams model output to the browser. This is the core of an AI Full-Stack role.

Learn in this order

1. [MUST] HTTP, REST, status codes, headers, JSON; test with curl and Postman (or Bruno)
2. [MUST] FastAPI core: routing, path and query parameters, request bodies, response models, dependency injection, middleware, CORS, exception handlers, auto-generated OpenAPI docs
3. [MUST] Pydantic v2: models, validation, settings (pydantic-settings), serialization
4. [MUST] SQL first, ORM second: PostgreSQL, joins, indexes, transactions, normalization; then SQLAlchemy 2.0 and Alembic migrations
5. [MUST] Authentication and authorization: password hashing (argon2 or bcrypt), JWT access and refresh tokens, role-based access, OAuth 2.0 login with Google or GitHub
6. [MUST] Testing with pytest (fixtures, TestClient, a separate test database), logging, structured error handling
7. [SHOULD] Redis: caching, rate limiting, token blocklists
8. [SHOULD] Background jobs: FastAPI BackgroundTasks for small work, then a queue (Celery, ARQ or RQ) for heavy work such as document processing
9. [SHOULD] Streaming: Server-Sent Events (SSE) for LLM tokens, WebSockets for two-way features like live collaboration
10. [OPTIONAL] GraphQL, gRPC, message brokers such as RabbitMQ or Kafka

Which framework when

- **FastAPI (your primary):** async, typed, automatic docs; the default for AI/ML APIs and what most AI startups use.
- **Flask:** minimal and flexible; good for tiny services and legacy codebases. Once you know FastAPI you can pick it up in a day.
- **Django:** batteries included (admin, ORM, auth, forms); best for content-heavy or CRUD-heavy products and companies already on it. Learn the basics after you are job-ready; many Python job listings still ask for it.

Practice

- After each numbered topic add one feature to the same API instead of starting new toy apps: CRUD, then validation, DB, auth, tests, caching, a background job
- Write 20+ pytest tests including failure cases (invalid token, duplicate user, forbidden role)
- SQL drills on PG Exercises until joins and GROUP BY feel automatic

Resources

- FastAPI tutorial (official): https://fastapi.tiangolo.com/tutorial/
- Pydantic docs: https://docs.pydantic.dev/latest/
- SQLAlchemy 2.0 tutorial: https://docs.sqlalchemy.org/en/20/tutorial/
- Alembic: https://alembic.sqlalchemy.org/
- PostgreSQL tutorial: https://www.postgresql.org/docs/current/tutorial.html
- PG Exercises: https://pgexercises.com/ and SQLBolt: https://sqlbolt.com/
- Redis docs: https://redis.io/docs/
- pytest docs: https://docs.pytest.org/
- MDN HTTP guide: https://developer.mozilla.org/en-US/docs/Web/HTTP
- OAuth 2.0 Simplified: https://www.oauth.com/
- Celery docs: https://docs.celeryq.dev/
- Postman Learning Center: https://learning.postman.com/
- Official FastAPI full-stack template (read it, do not clone blindly): https://github.com/fastapi/full-stack-fastapi-template
- Django: https://docs.djangoproject.com/ · Flask: https://flask.palletsprojects.com/

Build: Project 2, LedgerAPI.

Exit check: you can build a JWT-secured CRUD API with migrations and tests in a weekend, and explain what each layer does.

5. Phase 4: Frontend Development for AI Applications (Months 4–5, part-time)

Why: AI products live or die on interface details: streaming text, stop buttons, upload progress, citations. You already know the React fundamentals, so this phase is about TypeScript and AI-specific patterns.

What you reuse and what you skip

- Reuse: HTML, CSS, Tailwind, React components and props, Git, deploying a static site. Skip those tutorials entirely.
- Add: TypeScript, server-state management, typed API clients, auth against your own backend (instead of Firebase Auth), and streaming UI.

Learn

- [MUST] TypeScript: types vs interfaces, unions, generics (basic), `unknown` vs `any`, typing props, hooks and API responses; Zod for runtime validation
- [MUST] React hooks in depth: `useEffect` pitfalls, `useRef`, `useReducer`, custom hooks
- [MUST] Data fetching with TanStack Query; a typed API client (generate types from FastAPI's OpenAPI schema)
- [MUST] Auth integration: login and register forms, protected routes, token refresh, httpOnly cookies vs in-memory tokens
- [MUST] AI interface patterns: chat UI, streaming tokens with `fetch` + ReadableStream (or EventSource for SSE), a Stop button using AbortController, markdown and code rendering, loading, error and retry states
- [MUST] File upload with progress (FormData), document list with processing status (polling), citation display for RAG answers
- [SHOULD] State management with Zustand or Context, React Hook Form, responsive layouts and accessibility basics, Vite
- [OPTIONAL] Next.js (useful for SEO or landing pages; not required when FastAPI is your backend), Vercel AI SDK, shadcn/ui
- [ADVANCED] WebRTC voice interfaces, server components for AI streaming

Practice

- Convert one of your existing React projects to TypeScript with strict mode on
- Build a chat UI against a fake FastAPI endpoint that streams words with `asyncio.sleep`; add stop, retry and a typing indicator
- Build an upload page with drag-and-drop, progress bar and a status badge that polls the backend

Resources

- React docs: https://react.dev/learn
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Total TypeScript free tutorials: https://www.totaltypescript.com/tutorials

- TanStack Query: https://tanstack.com/query/latest
- Zod: https://zod.dev/
- Tailwind CSS docs: https://tailwindcss.com/docs
- Vite: https://vite.dev/
- MDN (fetch, streams, AbortController): https://developer.mozilla.org/
- javascript.info: https://javascript.info/
- shadcn/ui: https://ui.shadcn.com/

Build: a polished chat interface with streaming and file upload (it becomes the frontend of Project 3).

Exit check: every API response in your frontend is typed; you can explain how token streaming gets from the server to the screen.

6. Phase 5: Mathematics and Data Science Fundamentals (Month 6)

Why: You do not need a math degree to build AI applications, but you must understand vectors, probability and gradients well enough to read documentation, debug models and judge results.

Essential now [MUST]

- Linear algebra: vectors, dot product, cosine similarity (you will use this for embeddings), matrices, matrix multiplication, shapes and broadcasting
- Statistics and probability: mean, median, variance, standard deviation, normal distribution, conditional probability and Bayes' rule, correlation vs causation, sampling, train/test thinking
- Calculus intuition: derivative as slope, partial derivatives, chain rule (conceptual), gradient descent

Learn when needed [SHOULD]

- Eigenvalues, SVD and PCA math (when you study dimensionality reduction)
- Cross-entropy and KL divergence intuition (when you study loss functions in Phase 7)
- Hypothesis testing and confidence intervals (when you run experiments or A/B tests)

Skip until you need it [ADVANCED]: measure theory, advanced optimization theory, Bayesian inference in depth.

Tools

- [MUST] NumPy, Pandas, Matplotlib and Seaborn, Jupyter Notebook
- [MUST] Data cleaning (missing values, outliers, duplicates, types), preprocessing (scaling, encoding), exploratory data analysis (EDA)

Practice

- Implement dot product, matrix multiply and gradient descent for linear regression in pure NumPy before using any library
- Clean a messy CSV with Pandas and write down every decision you made
- Complete the Kaggle Learn Pandas and Data Visualization micro-courses

Resources

- 3Blue1Brown, Essence of Linear Algebra and Essence of Calculus: https://www.youtube.com/@3blue1brown
- StatQuest (statistics and ML explained simply): https://www.youtube.com/@statquest
- Khan Academy: https://www.khanacademy.org/math
- Mathematics for Machine Learning (free book): https://mml-book.com/
- NumPy absolute beginners guide: https://numpy.org/doc/stable/user/absolute_beginners.html
- pandas getting started: https://pandas.pydata.org/docs/getting_started/index.html
- Matplotlib: https://matplotlib.org/stable/tutorials/index.html · Seaborn: https://seaborn.pydata.org/tutorial.html
- Kaggle Learn (free, interactive): https://www.kaggle.com/learn

Build: an EDA notebook on a public dataset (Titanic or a house-prices dataset from Kaggle) with a written summary of findings, published on GitHub.

Exit check: you can explain gradient descent in plain words, compute cosine similarity by hand, and turn a messy CSV into a clean analysis-ready table.

7. Phase 6: Machine Learning (Months 6–7)

Why: Classic ML is still the right tool for tabular prediction, ranking and anomaly detection, and it teaches the evaluation discipline you will later apply to LLM systems.
Prerequisites: Phase 5 (NumPy, Pandas, basic statistics).

Learn

- [MUST] AI vs ML vs deep learning; supervised vs unsupervised learning; train/validation/test split; overfitting, underfitting, bias-variance
- [MUST] Regression and classification: linear and logistic regression; metrics (MAE, RMSE, R^2; accuracy, precision, recall, F1, ROC-AUC, confusion matrix)
- [MUST] Decision trees and random forests; know why ensembles beat single trees
- [MUST] Feature engineering, scaling, encoding, class imbalance, cross-validation, hyperparameter search, data leakage, scikit-learn Pipelines
- [MUST] Model deployment basics: save with joblib, serve with FastAPI, version the model, log predictions
- [SHOULD] KNN, Naive Bayes, SVM (know the intuition and when each fits); gradient boosting (XGBoost or LightGBM)
- [SHOULD] Clustering (k-means, DBSCAN) and PCA
- [OPTIONAL] MLflow experiment tracking, DVC; [ADVANCED] AutoML, advanced time-series forecasting

Practical datasets

- Beginner: Iris, Wine, Titanic (classification), California Housing (regression), both built into scikit-learn or on Kaggle
- Intermediate: Telco Customer Churn and Credit Card Fraud (imbalanced classes), SMS Spam Collection (text), MNIST digits
- Recommendation: MovieLens
- Find more: Kaggle Datasets and the UCI repository https://archive.ics.uci.edu/

Practice

- For every algorithm: fit it on a toy dataset, compare against a dummy baseline, plot feature importance or a decision boundary, and write two sentences on when you would use it
- Always start with a baseline; always split data before any preprocessing step
- Enter the Kaggle “Getting Started” competitions (Titanic, House Prices) and submit once

Resources

- scikit-learn user guide: https://scikit-learn.org/stable/user_guide.html
- scikit-learn MOOC (Inria, free): https://inria.github.io/scikit-learn-mooc/
- Kaggle Learn, Intro and Intermediate Machine Learning: https://www.kaggle.com/learn

- Machine Learning Specialization (Andrew Ng, Coursera, auditable for free): https://www.coursera.org/specializations/machine-learning-introduction
- StatQuest ML playlists: https://www.youtube.com/@statquest
- Book: *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow* (Géron), newest edition; the scikit-learn part is the relevant one
- XGBoost docs: https://xgboost.readthedocs.io/

Build: Project 4, PriceWise (a trained model behind a FastAPI endpoint).

Exit check: you can choose a metric that matches the business problem, explain data leakage with an example, and deploy a scikit-learn model as an API.

8. Phase 7: Deep Learning and Neural Networks (Months 7–8)

Why: You need working literacy in neural networks and transformers to use, fine-tune, evaluate and debug modern AI. You do not need to invent new architectures for application roles, so this phase is deliberately lean. **Prerequisites:** gradients and loss functions (Phase 5), train/test discipline (Phase 6).

Learn

- [MUST] The perceptron and neuron, activation functions (ReLU, sigmoid, softmax), forward propagation, loss functions (MSE, cross-entropy)
- [MUST] Backpropagation (understand it conceptually and build a tiny autograd once), optimizers (SGD, Adam), learning rate, batch size, dropout, weight decay, early stopping
- [MUST] PyTorch: tensors, autograd, `nn.Module`, Dataset and DataLoader, the training loop, saving and loading models, using a free GPU
- [MUST] Transformers: attention and self-attention, positional encoding, encoder vs decoder, how GPT-style models generate text token by token
- [MUST] Hugging Face basics: `transformers`, `datasets`, tokenizers, pipelines; fine-tune a small pretrained model on your own labelled data
- [SHOULD] CNNs for images (convolution, pooling) and transfer learning with a pretrained network
- [SHOULD] RNNs and LSTMs: understand the idea, the long-dependency problem and why transformers replaced them (no heavy implementation)
- [OPTIONAL] LoRA/PEFT fine-tuning, quantization

- [ADVANCED] Training from scratch, distributed training, CUDA programming, JAX/TensorFlow, diffusion models

Practice

- Train an MLP on MNIST, then a small CNN on CIFAR-10, then transfer learning on a small custom image dataset
- Fine-tune DistilBERT (or similar) for sentiment classification and compare it with a TF-IDF + logistic regression baseline
- Use Google Colab or Kaggle notebooks for free GPUs

Resources

- PyTorch tutorials: https://pytorch.org/tutorials/
- Andrej Karpathy, Neural Networks: Zero to Hero: https://karpathy.ai/zero-to-hero.html
- 3Blue1Brown neural networks series: https://www.youtube.com/@3blue1brown
- Dive into Deep Learning (free book): https://d2l.ai/
- fast.ai Practical Deep Learning: https://course.fast.ai/
- Hugging Face LLM Course: https://huggingface.co/learn/llm-course
- The Illustrated Transformer (Jay Alammar): https://jalammar.github.io/illustrated-transformer/
- Transformers docs: https://huggingface.co/docs/transformers
- Optional deeper reading: *Understanding Deep Learning* (Prince, free): https://udlbook.github.io/udlbook/

Build: two notebooks on GitHub: (a) an image classifier with transfer learning wrapped in a FastAPI endpoint, (b) a fine-tuned text classifier pushed to the Hugging Face Hub with a model card.

Exit check: you can explain in plain language what happens in one training step and what attention does, and you can fine-tune a small model without copying a notebook.

9. Phase 8: Generative AI and LLM Development (Month 5 for basics, Month 9 for depth)

Why: This is the highest-demand skill for AI Application Developers. Split it in two: **Part A** (API use) needs only Python, FastAPI and basic prompting; **Part B** (RAG, agents, evaluation) comes after you have the ML mindset.

Part A: LLM API fundamentals (Month 5)

- [MUST] What an LLM is from a user's view: tokens, context windows, temperature, cost and latency trade-offs, why models hallucinate
- [MUST] Prompt engineering: system prompts, clear instructions, examples (few-shot), delimiters, asking for step-by-step reasoning where useful, iterating with test cases
- [MUST] One provider API in depth (OpenAI, Anthropic Claude or Google Gemini); the others have the same shape, so switching later is easy
- [MUST] Streaming responses, structured outputs (JSON schema validated with Pydantic), function and tool calling
- [MUST] Production hygiene: retries with backoff, timeouts, rate limits, cost and token tracking, API keys only on the server
- [SHOULD] Multimodal input (images, PDFs), Hugging Face and local models via Ollama

Part B: Retrieval, agents and quality (Month 9)

- [MUST] Embeddings and semantic search; cosine similarity (from Phase 5)
- [MUST] RAG pipeline: load documents, chunk (size, overlap, structure-aware), embed, store, retrieve (top-k, metadata filters), assemble the prompt, return citations
- [MUST] Vector stores: start with Chroma or FAISS locally, then **pgvector** inside PostgreSQL (one database to run). Pinecone is [OPTIONAL] when you need a managed service
- [MUST] Evaluation: build a test set of 30–50 questions with expected sources; measure retrieval hit rate and answer faithfulness; use LLM-as-judge carefully and spot-check by hand; re-run after every prompt or chunking change
- [MUST] Guardrails and hallucination reduction: answer only from retrieved context, say "I don't know", require citations, validate outputs against a schema, treat retrieved documents as untrusted (prompt injection), redact personal data, set usage and cost limits
- [SHOULD] Hybrid search (keyword + vector) and reranking; agents (tool-calling loop, memory, human approval steps); LangGraph for stateful agent workflows; LangChain for loaders and integrations; Model Context Protocol (MCP) basics
- [OPTIONAL] LlamaIndex, fine-tuning fundamentals (only when prompting and RAG fail; try LoRA), speech and vision pipelines
- [ADVANCED] Pretraining, RLHF, distillation, self-hosted inference servers

Essential vs optional in one line: essential is SDK + prompting + structured output + tool calling + RAG + pgvector or Chroma + evals + guardrails. Everything framework-specific is optional until a job requires it. Learn the raw SDK before LangChain; often plain Python is simpler.

Practice

- Part A: build a CLI chatbot with history, then add streaming, then a tool call (weather or calculator), then a Pydantic-validated JSON extraction task
- Part B: index 20 PDFs, write 40 test questions, measure retrieval, change chunk size, measure again, and record what changed
- Break your own system: write 10 adversarial prompts (prompt injection, off-topic, missing answer) and make the app handle them

Resources

- OpenAI API docs: https://platform.openai.com/docs
- Claude docs: https://docs.claude.com/ and prompt engineering overview: https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
- Gemini API docs: https://ai.google.dev/gemini-api/docs
- LangChain docs: https://docs.langchain.com/ · LangGraph: https://langchain-ai.github.io/langgraph/
- Chroma: https://docs.trychroma.com/ · FAISS: https://github.com/facebookresearch/faiss · pgvector: https://github.com/pgvector/pgvector · Pinecone: https://docs.pinecone.io/
- Ragas (RAG evaluation): https://docs.ragas.io/
- Ollama (local models): https://ollama.com/
- Model Context Protocol: https://modelcontextprotocol.io/
- DeepLearning.AI short courses (free): https://www.deeplearning.ai/short-courses/
- Andrej Karpathy on YouTube (Intro to LLMs, Let's build GPT): https://www.youtube.com/@AndrejKarpathy
- Book: *AI Engineering* (Chip Huyen, O'Reilly 2025), the best practical overview of building on foundation models

Build: Project 3 (Part A); Projects 5, 6 and 11 (Part B).

Exit check: you can build a cited, evaluated RAG app in a week and show a number that proves a change made it better.

10. Phase 9: AI Full-Stack Application Development (Month 10)

Why: This phase turns separate skills into one production-style system. **Prerequisites:** Phases 3, 4 and 8.

Target stack: React + TypeScript (Vite) → FastAPI → PostgreSQL with pgvector → Redis → worker process → object storage → LLM API, all running in Docker Compose.

Data flow 1: chat with your documents

1. The browser (React + TypeScript) sends the message with the user's token to a FastAPI streaming endpoint (SSE).
2. FastAPI validates the token with a dependency, checks the rate limit in Redis and loads the conversation from PostgreSQL.
3. The query is embedded; a vector search runs with a filter on user or tenant ID; the top chunks come back (optionally reranked).
4. The prompt is assembled (system instructions, retrieved context, recent history) and sent to the LLM API with streaming on.
5. Tokens stream back through FastAPI to the browser, which renders them incrementally; citations arrive at the end.
6. The finished answer, citations, token usage and latency are saved to PostgreSQL and logged.

Data flow 2: document ingestion

1. The browser uploads a file to FastAPI; the file goes to object storage (S3, Cloudflare R2 or a local volume in development); a database row is created with status `queued`.
2. A worker (Celery, ARQ or RQ, using Redis as the queue) picks up the job, extracts text, chunks it, embeds in batches and writes vectors with metadata; the status becomes `ready` or `failed` with an error message.
3. The frontend polls the status endpoint (or listens on SSE) and shows progress.

Cross-cutting concerns

- Authentication: JWT with refresh; every query filters by owner ID so one user can never see another's documents
- Configuration through environment variables; secrets never in Git
- Services in Compose: web, api, worker, postgres (pgvector), redis, plus a reverse proxy such as Caddy or Nginx

- Observability: structured logs, request IDs, error tracking, per-request token and cost records

Design rules

- Never call the LLM from the browser; keys stay on the server
- Put the LLM provider behind one interface (`services/llm.py`) so you can swap models
- Keep prompts in versioned files, not scattered strings
- Make jobs idempotent so retries are safe
- Use PostgreSQL for everything (including vectors) until measured load says otherwise

Suggested layout: `backend/app/{api, core, models, schemas, services, workers}` and `backend/tests`; `frontend/src/{components, features, hooks, lib}`; a root `docker-compose.yml`, `.env.example` and a README with an architecture diagram.

Practice

- Bring up all five services with one `docker compose up`
- Draw the architecture in Excalidraw and put the image in your README
- Run a small load test (for example with k6 or Locust) and fix the first bottleneck you find

Resources

- Docker Compose docs: https://docs.docker.com/compose/
- The Twelve-Factor App: https://12factor.net/
- System Design Primer: https://github.com/donnemartin/system-design-primer
- Official FastAPI full-stack template (for structure ideas): https://github.com/fastapi/full-stack-fastapi-template
- Excalidraw: https://excalidraw.com/

Build: Project 8 (KnowledgeHub), then Project 12 (FocusAI capstone).

Exit check: you can draw the architecture from memory and explain where each failure (LLM timeout, worker crash, expired token) is handled.

11. Phase 10: Deployment, DevOps and Production (Month 11)

Why: An app that only runs on your laptop is not a portfolio piece or a job skill. Employers want to see a live URL, a CI pipeline and evidence you think about security and cost.

Learn

- [MUST] Linux and command line: navigation, permissions, processes, ssh, grep, curl, jq, basic bash scripts
- [MUST] Docker: good Dockerfiles, multi-stage builds, volumes, networks; Docker Compose
- [MUST] GitHub Actions: on every push run lint, type check and tests; on merge build and deploy
- [MUST] Environment variables and secrets: `.env` locally, platform secret stores in production, rotate leaked keys immediately
- [MUST] Deployment: Vercel for the frontend; Render or Railway for the API, worker, PostgreSQL and Redis (the quickest path to a live URL)
- [MUST] API security: input validation, CORS, HTTPS, rate limiting, per-user usage caps for LLM calls, the OWASP API Security Top 10
- [SHOULD] AWS fundamentals: IAM, EC2, S3, RDS, a managed container service (ECS/Fargate) or Lambda overview, CloudWatch, **billing alarms**
- [SHOULD] Monitoring and logging: structured logs, Sentry for errors, health-check endpoints, uptime monitoring, LLM tracing (for example Langfuse)
- [SHOULD] Performance and scalability: database indexes, pagination, connection pooling, caching, batching embedding calls, stateless API servers, queues for slow work
- [SHOULD] Database backups: scheduled `pg_dump` or managed backups, and a tested restore
- [OPTIONAL] Prometheus and Grafana; [ADVANCED] Kubernetes, Terraform, service meshes (learn only when a job requires them)

Practice

- Dockerize Project 2 and add a CI workflow that fails on a broken test
- Deploy Project 8 end to end with a custom or platform URL
- Restore a database from a backup into a fresh instance; set an AWS billing alarm; run a dependency vulnerability scan

Resources

- Linux Journey: https://linuxjourney.com/
- The Missing Semester of Your CS Education (MIT): https://missing.csail.mit.edu/
- Docker get started: https://docs.docker.com/get-started/
- GitHub Actions docs: https://docs.github.com/actions
- Render docs: https://render.com/docs · Railway docs: https://docs.railway.com/ · Vercel docs: https://vercel.com/docs
- AWS Skill Builder (free cloud fundamentals courses): https://skillbuilder.aws/
- OWASP API Security Project: https://owasp.org/API-Security/
- Sentry docs: https://docs.sentry.io/ · Langfuse docs: https://langfuse.com/docs
- roadmap.sh DevOps (checklist): https://roadmap.sh/devops

Build: a live, CI-deployed Project 8 with monitoring, backups and a short "Architecture and security" section in the README.

Exit check: a stranger can open your live URL; a failing test blocks a deploy; you know what your monthly cloud bill will be.

12. Project-Based Learning: 13 Projects

How to use this list. Projects marked ★ are **core** (8 of them): together they are the minimum portfolio that proves you can build, ship and evaluate. The rest are **stretch** projects for extra depth, freelancing or the ML Engineer track. Build the core ones in the order shown; do not start a stretch project before the previous core project is deployed or at least documented. Estimates assume 3 h/day.

Beginner

Project 1: DeskAutomator ★ (Month 1)

- **Overview:** A command-line tool that organizes a messy folder, renames files by rules and produces a weekly report.
- **Real-world problem:** People waste hours sorting downloads, invoices and screenshots by hand; small offices need repeatable cleanup.
- **Technologies:** Python, pathlib, Typer or argparse, openpyxl, cron or `schedule`, pytest
- **Features:** rules in a YAML config, dry-run mode, undo log, duplicate detection by file hash, Excel/CSV report, scheduled runs

- **Backend architecture:** no server; modules for scanner, rules engine, actions, reporter and config loader; logging
- **AI/ML implementation:** none required. Stretch in Month 5: an LLM suggests folder names for files no rule matches
- **Database design:** SQLite table `actions(id, path_old, path_new, file_hash, timestamp)` powering the undo feature
- **Difficulty and time:** Beginner · 2–3 weeks (about 30 hours)
- **GitHub presentation:** README with a before/after GIF, install via pipx, test badge, tagged v1.0 release, CHANGELOG

Project 2: LedgerAPI ★ (Months 3–4)

- **Overview:** A multi-user expense tracker REST API with a small React + TypeScript dashboard.
- **Real-world problem:** Personal and small-business finance tracking with secure per-user data, reports and budgets.
- **Technologies:** FastAPI, Pydantic, PostgreSQL, SQLAlchemy, Alembic, JWT, Redis, pytest, Docker
- **Features:** register, login and refresh; transaction CRUD; categories; monthly summary; budget alerts; CSV import and export; rate limiting
- **Backend architecture:** routers → services → repositories; dependency-injected DB session and current user; background task for CSV import; Redis cache for summaries
- **AI/ML implementation:** optional, added after Month 5: an LLM auto-categorizes transactions using structured output
- **Database design:** `users`, `categories`, `transactions(user_id, category_id, amount, date, note)`, `budgets`; index on `(user_id, date)`
- **Difficulty and time:** Beginner to Intermediate · 5–6 weeks
- **GitHub presentation:** OpenAPI screenshot, Postman collection, coverage badge (80%+), one-command Docker Compose run, ER diagram

Project 3: StreamChat ★ (Month 5)

- **Overview:** A ChatGPT-style chatbot with saved conversations, streaming answers and model selection.
- **Real-world problem:** Teams want a private chat interface over an LLM with history and controlled cost.

- **Technologies:** React + TypeScript, FastAPI (SSE), PostgreSQL, Redis, one or two LLM APIs
- **Features:** login, streamed answers, stop button, conversation list (rename, delete), system-prompt presets, markdown and code rendering, token usage per conversation, daily message limit
- **Backend architecture:** a streaming `/chat` endpoint; a provider wrapper with retries and timeouts; history trimming or summarization to fit the context window
- **AI/ML implementation:** prompt design, context-window management, one tool call (calculator or web fetch), generated conversation titles
- **Database design:** `users`, `conversations`, `messages(role, content, tokens_in, tokens_out, model)`, `usage_daily`
- **Difficulty and time:** Beginner to Intermediate · 3–4 weeks
- **GitHub presentation:** demo GIF of streaming and stop, deployed link, a short design note on context management and cost

Intermediate

Project 4: PriceWise ★ (Months 6–7)

- **Overview:** An ML service that predicts house prices (or customer churn) and explains the result through a web form.
- **Real-world problem:** Businesses need transparent, deployable predictions rather than notebooks.
- **Technologies:** pandas, scikit-learn (XGBoost optional), joblib, FastAPI, Pydantic, PostgreSQL, React, Docker
- **Features:** training script with a pipeline, metrics report, `/predict` and `/predict/batch`, input validation, feature-importance explanation, model-version endpoint, prediction log
- **Backend architecture:** training code separate from serving code; versioned model artifact loaded at startup; predictions logged in the background
- **AI/ML implementation:** baseline vs tuned models, cross-validation, leakage checks, error analysis by segment
- **Database design:** `predictions(input_json, output, model_version, created_at)`, `models(version, metrics_json, trained_at)`
- **Difficulty and time:** Intermediate · 4–5 weeks

- **GitHub presentation:** notebook plus production code, a model card (data, metrics, limitations), a three-model comparison table, live demo

Project 5: AskMyPDF ★ (Month 9)

- **Overview:** Upload PDFs and ask questions; every answer cites the page it came from.
- **Real-world problem:** Reading long contracts, papers and manuals is slow; people need answers they can verify.
- **Technologies:** FastAPI, React + TypeScript, PyMuPDF or pypdf, embeddings API, Chroma then pgvector, PostgreSQL, a Redis-backed queue, LLM API
- **Features:** upload with processing status, per-document chat, page citations with the matching snippet, an explicit “not found in this document” answer, delete document and its vectors
- **Backend architecture:** ingestion worker (extract, chunk, embed, store) and query path (embed, retrieve, prompt, stream); object storage for files
- **AI/ML implementation:** chunking with overlap, top-k with metadata filter, grounded prompt, faithfulness check on a 30-question test set
- **Database design:** `documents(owner_id, status)`, `chunks(document_id, page, text, embedding)`, `conversations`, `messages`
- **Difficulty and time:** Intermediate · 3–4 weeks
- **GitHub presentation:** evaluation table in the README, short demo GIF, a “known failure cases” section

Project 6: ResumeLens (Month 9, stretch)

- **Overview:** Compares a resume with a job description and returns a score, skill gaps and suggestions.
- **Real-world problem:** Applicants do not know why resumes get filtered out; recruiters need consistent screening.
- **Technologies:** FastAPI, PDF/DOCX parsing, LLM with structured outputs, embeddings, React, PostgreSQL
- **Features:** resume upload, pasted job description, extracted skills and experience, match score with breakdown, keyword gaps, bullet-point rewrite suggestions, history
- **Backend architecture:** parse, normalize, extract with a Pydantic schema, score (embedding similarity plus explicit skill matching), generate report
- **AI/ML implementation:** structured extraction, hybrid scoring, a rule that the model must never invent experience, a fairness and limitations note

- **Database design:** `users`, `resumes(parsed_json)`, `jobs`, `analyses(score, report_json)`
- **Difficulty and time:** Intermediate · 2 weeks
- **GitHub presentation:** anonymized sample outputs, a scoring-methodology page, a privacy statement with data deletion

Project 7: BookMatch (Month 8, stretch; recommended for the ML Engineer track)

- **Overview:** A recommendation engine for books or movies using content embeddings and collaborative filtering.
- **Real-world problem:** Users face too many options; platforms need personalised ranking.
- **Technologies:** pandas, scikit-learn, sentence-transformers, FastAPI, PostgreSQL with pgvector, React
- **Features:** rate items, similar-items view, personalised list, cold-start fallback (popular plus content-based), “because you liked X” explanations
- **Backend architecture:** an offline job computes embeddings and factors; the online API serves top-N from precomputed tables; Redis cache
- **AI/ML implementation:** content-based (embedding cosine), item-item and matrix factorization on MovieLens, evaluation with precision@k and recall@k on a time-based split
- **Database design:** `items(embedding)`, `ratings(user_id, item_id, value, ts)`, `recommendations_cache`
- **Difficulty and time:** Intermediate · 3 weeks
- **GitHub presentation:** offline-evaluation chart, comparison of approaches, demo with five sample users

Advanced

Project 8: KnowledgeHub ★ (Month 10)

- **Overview:** A multi-user RAG knowledge base: upload documents, ask questions with citations, control who sees what.
- **Real-world problem:** Company knowledge is scattered across PDFs, wikis and notes; employees cannot find answers or trust them.
- **Technologies:** React + TypeScript, FastAPI, PostgreSQL with pgvector, Redis, Celery or ARQ, S3-compatible storage, LLM + embeddings, Docker Compose, GitHub Actions

- **Features:** workspaces and roles, folders and tags, ingestion for PDF/DOCX/Markdown/URL, hybrid search, chat with citations, thumbs-up/down feedback, admin usage dashboard, re-index, deletion that also removes vectors
- **Backend architecture:** the two data flows from Phase 9; permission filters applied inside the retrieval query; audit log
- **AI/ML implementation:** hybrid retrieval and reranking, query rewriting, an evaluation suite that runs in CI with a minimum retrieval hit rate, prompt-injection defences, per-workspace cost limits
- **Database design:** `users`, `workspaces`, `memberships`, `documents`, `chunks(embedding, tsvector)`, `conversations`, `messages`, `feedback`, `usage_events`, `audit_log`
- **Difficulty and time:** Advanced · 6–8 weeks
- **GitHub presentation:** architecture diagram, live demo with seeded public documents, evaluation report, an `adr/` folder of decision records, a security section

Project 9: DocFlow (stretch)

- **Overview:** A pipeline that ingests invoices or forms, extracts fields into structured data, validates them and routes uncertain ones to human review.
- **Real-world problem:** Manual data entry from documents is slow and error-prone.
- **Technologies:** FastAPI, Celery or ARQ, PostgreSQL, S3 storage, OCR (Tesseract or a cloud OCR) or a multimodal LLM, Pydantic, React review UI
- **Features:** batch upload, field extraction with confidence, validation rules (totals add up, dates valid), review queue with side-by-side view, export to CSV/JSON/webhook, audit trail
- **Backend architecture:** upload, classify, OCR or vision, extract, validate, review, export; idempotent jobs and a dead-letter queue
- **AI/ML implementation:** schema-constrained extraction, confidence heuristics, field-level accuracy against a labelled sample
- **Database design:** `documents`, `extractions(field, value, confidence, status)`, `review_tasks`, `schemas`, `audit_events`
- **Difficulty and time:** Advanced · 5–6 weeks
- **GitHub presentation:** per-field accuracy table, a measured time-saved estimate, demo video

Project 10: SupportPilot (stretch)

- **Overview:** An AI customer-support platform: a chat widget answers from help-centre content, creates tickets and hands over to a human.
- **Real-world problem:** Support teams answer the same questions repeatedly; customers want instant, accurate replies.
- **Technologies:** FastAPI, WebSockets, PostgreSQL with pgvector, Redis, React widget and agent dashboard, LLM tool calling
- **Features:** embeddable widget, cited RAG answers, ticket-creation tool, urgency tagging, human takeover, analytics (deflection rate, satisfaction)
- **Backend architecture:** widget, then a WebSocket gateway, a conversation service, retrieval plus LLM with tools (`create_ticket`, `lookup_order`), and Redis pub/sub to the agent dashboard
- **AI/ML implementation:** tool calling with confirmation steps, escalation when confidence is low, guardrails for refunds and policy, evaluation on a transcript set
- **Database design:** `tenants`, `customers`, `conversations`, `messages`, `tickets`, `kb_articles`, `kb_chunks`, `events`
- **Difficulty and time:** Advanced · 6 weeks
- **GitHub presentation:** widget demo on a sample site, analytics screenshots, documented guardrails

Professional

Project 11: ResearchCrew ★ (Month 11, 2–3 weeks)

- **Overview:** A multi-agent app where planner, researcher, writer and critic agents produce a cited report on a topic.
- **Real-world problem:** Research and synthesis takes hours; teams need summaries they can trace to sources.
- **Technologies:** LangGraph (or a plain Python state machine), FastAPI, SSE, PostgreSQL, search and fetch tools, React
- **Features:** topic input, plan approval step (human in the loop), streamed progress timeline, source list, editable report, Markdown/PDF export, run history with cost per run
- **Backend architecture:** a graph of nodes sharing typed state; checkpoints in PostgreSQL; tool layer with timeouts and a domain allow-list; step and cost budgets

- **AI/ML implementation:** tool calling, a critic loop, citation checking (each claim maps to a source), evaluation on 10 topics against a single-prompt baseline
- **Database design:** `runs`, `steps(node, input, output, tokens)`, `sources`, `reports`
- **Difficulty and time:** Professional · 2–3 weeks
- **GitHub presentation:** graph diagram, an honest "multi-agent vs single prompt" comparison, trace screenshots

Project 12: FocusAI ★ (Months 11–12, capstone)

- **Overview:** A productivity platform with notes, tasks and a planner, plus an AI assistant that searches your content, summarizes and acts through tools.
- **Real-world problem:** Knowledge workers juggle notes, tasks and meetings across tools and lose context.
- **Technologies:** the full Phase 9 stack plus a LangGraph agent, workers, Docker, CI/CD, Sentry
- **Features:** notes with semantic search, tasks with AI-extracted action items, weekly plan generation, "ask my workspace" chat with citations, a tool-calling assistant (create task, schedule), usage limits, data export
- **Backend architecture:** modular monolith (auth, notes, tasks, ai modules), worker for embeddings and summaries, event table for audit, feature flags for AI features
- **AI/ML implementation:** RAG over user content, a tool-calling agent with confirmations, structured extraction, evaluations and cost tracking
- **Database design:** `users`, `notes`, `note_chunks(embedding)`, `tasks`, `plans`, `conversations`, `messages`, `tool_calls`, `usage_events`
- **Difficulty and time:** Professional · 6–8 weeks (scope it to an MVP and reuse code from Projects 8 and 11)
- **GitHub presentation:** live URL, a 2-minute demo video, architecture diagram, roadmap in GitHub Issues, CI badge, a case-study write-up

Project 13: ContentForge, an AI SaaS (stretch; freelance-ready)

- **Overview:** A multi-tenant AI SaaS (for example marketing copy and email drafts) with subscription billing and usage metering.
- **Real-world problem:** Small businesses need AI writing tools that match their brand voice.
- **Technologies:** FastAPI, React + TypeScript, PostgreSQL, Redis, Stripe in test mode, LLM APIs, Docker, GitHub Actions

- **Features:** organizations and seats, brand-voice profiles with examples, templates, generation history, plan limits and usage meter, Stripe checkout and webhooks, admin panel
- **Backend architecture:** `org_id` on every tenant row; a billing module that handles webhooks idempotently; usage-metering middleware; a job queue for long generations
- **AI/ML implementation:** prompt templates with variables, brand-voice few-shot retrieval, output moderation, A/B prompt comparison
- **Database design:** `orgs`, `memberships`, `subscriptions`, `usage_events`, `brand_profiles`, `generations`, `templates`
- **Difficulty and time:** Professional · 5–6 weeks (reuse FocusAI foundations)
- **GitHub presentation:** pricing page, test-mode checkout demo, prompt A/B results, a "how billing works" document

13. The 12-Month Learning Plan

Daily template (3 hours, 70/20/10)

- **Theory, 35 min:** one concept from the docs or a video, with handwritten or typed notes in your own words
- **Coding, 125 min:** exercises first, then project work; on DSA days, 30 of these minutes go to problems
- **Revision, 20 min:** flashcards, reread yesterday's notes, re-run yesterday's code from memory
- **Weekly rhythm:** six study days; Sunday is a 1-hour weekly review (fill the tracking template in Section 18) and then rest
- **Job search timing:** update your resume in Month 9 and start applying for internships and junior roles in Month 10. Do not wait until you feel finished.

Month 1: Python fundamentals and Git (Phase 1)

- **Topics:** syntax, control flow, functions, collections, OOP, exceptions, file I/O, virtual environments, Git workflow
- **Weekly milestones:** W1 variables, conditions, loops, functions · W2 collections, modules, files, exceptions · W3 OOP, dataclasses, project structure, first pytest tests · W4 finish and release Project 1
- **Daily focus:** one concept per day, then 2–3 exercises on Exercism or CS50P

- **Coding practice:** 60 small exercises over the month; 4 mini-programs (guesser, contact book, bank account, log analyzer)
- **Project milestone:** DeskAutomator v1.0 tagged on GitHub by day 28
- **Outcome:** you write a 150-line CLI tool from scratch with tests, a README and a clean Git history

Month 2: Advanced Python and data structures (Phase 2)

- **Topics:** comprehensions, generators, decorators, context managers, type hints, asyncio, arrays, hash maps, stacks, queues, linked lists, trees, recursion, Big-O
- **Weekly milestones:** W1 comprehensions, generators; arrays and hash maps · W2 decorators, context managers, typing; stacks, queues, two pointers · W3 asyncio and httpx; linked lists, trees, recursion · W4 async tool with tests; BFS/DFS
- **Daily focus:** 30–40 min of problems, then the day's concept
- **Coding practice:** about 50 NeetCode problems (Easy to Medium); re-implement `@timer` and `@retry` from memory
- **Project milestone:** an async API health-checker, fully typed, mypy clean, tested
- **Outcome:** you read and write idiomatic, typed, async Python and state the complexity of your solutions

Month 3: Backend core with FastAPI, SQL and auth (Phase 3, part 1)

- **Topics:** HTTP and REST, FastAPI, Pydantic, PostgreSQL and SQL, SQLAlchemy, Alembic, password hashing, JWT, role checks, pytest for APIs
- **Weekly milestones:** W1 REST design, FastAPI CRUD in memory, Pydantic · W2 PostgreSQL, SQL drills, SQLAlchemy and migrations · W3 registration, login, refresh tokens, protected routes, ownership checks · W4 API test suite; Project 2 backend feature complete
- **Daily focus:** add one feature to LedgerAPI every day or two
- **Coding practice:** 5 SQL questions per day on PG Exercises; DSA on four days a week
- **Project milestone:** LedgerAPI with auth, CRUD, summaries and 20+ tests
- **Outcome:** you build a secured database-backed API with migrations and tests without copying a tutorial

Month 4: Backend completion and TypeScript frontend (Phase 3, part 2, and Phase 4, part 1)

- **Topics:** Redis caching and rate limiting, background jobs, logging, SSE and WebSockets, strict TypeScript, TanStack Query, typed API client, auth flow in React
- **Weekly milestones:** W1 Redis, background tasks, logging, error handling · W2 SSE and WebSockets; convert a React project to TypeScript · W3 TanStack Query, typed client from OpenAPI, login and protected routes · W4 React dashboard for LedgerAPI, Docker Compose, README, Postman collection
- **Daily focus:** alternate backend and frontend days; keep 30 min for DSA
- **Coding practice:** 10 more problems; rewrite one earlier endpoint test-first
- **Project milestone:** Project 2 finished end to end and running with one command
- **Outcome:** you own a full-stack CRUD app with your own auth, replacing what Firebase used to do for you

Month 5: AI interface and LLM fundamentals (Phase 4, part 2, and Phase 8, part A)

- **Topics:** LLM concepts, prompt design, provider API, streaming, structured outputs, tool calling, cost and error handling, streaming chat UI
- **Weekly milestones:** W1 first API calls, prompts, a CLI chatbot with history · W2 structured outputs with Pydantic, tool calling, retries, token and cost logging · W3 React chat UI with streaming, stop button, markdown · W4 persistence, rate limits, first public deploy of Project 3
- **Daily focus:** build from a blank file; compare your prompts on 10 saved test inputs
- **Coding practice:** a prompt test script that runs your 10 inputs and prints results side by side
- **Project milestone:** StreamChat live with a README and demo GIF
- **Outcome:** your first deployed AI product; you can explain tokens, context and streaming

Month 6: Mathematics, data science and ML start (Phase 5 and Phase 6, part 1)

- **Topics:** vectors, matrices, probability, statistics, NumPy, Pandas, visualization, EDA, scikit-learn basics, metrics

- **Weekly milestones:** W1 linear algebra and NumPy (gradient descent from scratch) · W2 Pandas, cleaning, EDA, statistics · W3 regression, classification, train/test, baselines, metrics · W4 publish the EDA notebook; submit to Titanic; prepare PriceWise data
- **Daily focus:** theory is heavier this month (up to 45 min) because the math has to land
- **Coding practice:** Kaggle Learn micro-courses on Pandas and visualization; one dataset analysed per week
- **Project milestone:** EDA notebook with written findings on GitHub
- **Outcome:** you can clean and explore a tabular dataset and train a baseline model correctly

Month 7: Machine learning completion and PyTorch start (Phase 6, part 2, and Phase 7, part 1)

- **Topics:** trees, forests, boosting, feature engineering, pipelines, cross-validation, model serving, neural network basics, backpropagation, PyTorch training loop
- **Weekly milestones:** W1 tree models, feature engineering, pipelines, cross-validation · W2 PriceWise training, evaluation, FastAPI serving, React form · W3 neural networks, backpropagation, PyTorch tensors and training loop · W4 MNIST MLP, CIFAR-10 CNN; finish Project 4
- **Daily focus:** compare every model against a baseline and write down why one wins
- **Coding practice:** Karpathy's Zero to Hero lessons 1–2 coded along; DSA reduced to three days a week
- **Project milestone:** PriceWise live with model card
- **Outcome:** you deploy an ML model as an API and explain one full training step

Month 8: Deep learning, transformers and embeddings (Phase 7, part 2)

- **Topics:** CNNs and transfer learning, RNN/LSTM concepts, attention and transformers, Hugging Face, fine-tuning, sentence embeddings, recommendation basics
- **Weekly milestones:** W1 transfer learning on a custom image set, wrapped in an API · W2 transformers, attention, Hugging Face course chapters · W3 fine-tune a small text classifier and push it to the Hub · W4 BookMatch with offline evaluation
- **Daily focus:** reading for understanding in the morning block, code in the long block
- **Coding practice:** reproduce a Hugging Face example without copying; plot learning curves

- **Project milestone:** two notebooks on GitHub, a Hub model with a model card, BookMatch metrics
- **Outcome:** you can fine-tune and evaluate a small model and you understand what embeddings are good for

Month 9: RAG, agents, evaluation, guardrails (Phase 8, part B)

- **Topics:** chunking, embeddings, vector stores, pgvector, citations, hybrid search, evaluation sets, prompt injection, LangGraph basics
- **Weekly milestones:** W1 first RAG script with Chroma and a test set · W2 pgvector, ingestion worker, AskMyPDF backend · W3 citations in the UI, hybrid search, guardrail and injection tests · W4 LangGraph basics with a small tool-using agent; optional ResumeLens
- **Daily focus:** after every change to chunking or prompts, re-run the evaluation and log the number
- **Coding practice:** write 40 test questions with expected sources; 10 adversarial prompts
- **Project milestone:** AskMyPDF deployed with an evaluation table in the README
- **Outcome:** you can prove with numbers that your RAG system improved

Month 10: Production AI architecture, KnowledgeHub (Phase 9)

- **Topics:** system design for AI apps, Docker Compose with five services, multi-tenant permissions, ingestion at scale, hybrid retrieval with filters, evaluation in CI
- **Weekly milestones:** W1 ERD, API contract, architecture diagram, Compose stack, auth and workspaces · W2 ingestion pipeline with worker and object storage · W3 permission-aware retrieval, streaming chat, citations · W4 CI evaluation gate, usage tracking, polish
- **Scope note:** the MVP is PDF and Markdown ingestion, workspaces, hybrid search, citations and an evaluation gate; DOCX, URL ingestion and the admin dashboard become GitHub Issues for later
- **Daily focus:** finish one vertical slice per day (database to API to UI)
- **Coding practice:** one system-design question per week, answered in writing
- **Project milestone:** KnowledgeHub MVP running locally; resume updated; first applications sent
- **Outcome:** you can draw and defend a production AI architecture

Month 11: Deployment, DevOps, security and agents (Phase 10 and Project 11)

- **Topics:** Linux and CLI, Docker best practices, GitHub Actions, deployment, secrets, monitoring, backups, API security, AWS basics, multi-agent graphs
- **Weekly milestones:** W1 Dockerfiles, CI pipeline for KnowledgeHub · W2 deploy to Render or Railway plus Vercel, Sentry, backups, rate limits, security checklist · W3 AWS fundamentals (IAM, S3, billing alarm); start ResearchCrew · W4 finish a lean ResearchCrew (graph, approval step, citations); write the FocusAI design document
- **Daily focus:** automate every manual step you did twice
- **Coding practice:** break and restore the database; add rate limiting; run a dependency scan
- **Project milestone:** KnowledgeHub live with CI/CD and monitoring; ResearchCrew working
- **Outcome:** a live URL, a deploy pipeline and a security checklist you can show

Month 12: Capstone and career launch

- **Topics:** product scoping, reuse of earlier modules, portfolio polish, resume, LinkedIn, interview practice, freelancing profile
- **Weekly milestones:** W1–2 FocusAI MVP (reuse auth, RAG and agent code) and deploy · W3 README rewrites, demo videos, case studies, GitHub profile, resume and LinkedIn · W4 mock interviews, 10–15 applications per week, networking messages, freelance profile
- **Daily focus:** 90 min building or polishing, 60 min interview preparation, 30 min applications or outreach
- **Coding practice:** two timed DSA problems and one system-design answer per day on interview prep days
- **Project milestone:** FocusAI live; all core projects have polished READMEs and demos
- **Outcome:** job-ready portfolio and an active application pipeline; take the final assessment in Section 19

14. Career Preparation

Market reality. Junior roles are competitive. Expect to send dozens of applications and to hear back from a small fraction. What moves the odds most: deployed projects with evidence of quality, clear READMEs, a focused profile, and applying early and steadily.

AI Application Developer vs ML Engineer

- **AI Application Developer:** builds products on top of existing models (LLM APIs, RAG, agents). Cares about reliability, latency, cost, evaluation and user experience. Stack: Python and TypeScript, FastAPI, PostgreSQL, vector search, LLM SDKs. Math needs are light. Output: shipped features.
- **ML Engineer:** trains, fine-tunes, optimizes and deploys models and the data and training pipelines around them. Cares about data quality, experiments, model performance, serving and monitoring. Needs stronger statistics and math, more PyTorch, and often more formal credentials. Output: models and ML infrastructure.
- **Overlap:** evaluation, deployment, data handling. Starting as an application developer and moving toward ML later is a common and realistic path.

Role 1: Python Backend Developer

- **Required skills:** Phases 1–3 and 10: Python, FastAPI or Django, SQL and PostgreSQL, auth, testing, Docker, CI, basic cloud; DSA at Easy/Medium level
- **Portfolio:** Project 2 and Project 8, both deployed, with tests and CI
- **Titles to search:** Junior Python Developer, Backend Engineer (Python), API Developer

Role 2: AI Application Developer

- **Required skills:** everything in Role 1 plus Phase 8: prompting, structured output, tool calling, RAG, vector search, evaluation, guardrails, cost control
- **Portfolio:** Projects 3, 5, 8 and 11 with evaluation numbers in the READMEs
- **Titles to search:** AI Engineer (application focus), LLM Application Developer, Generative AI Developer, Software Engineer (AI)

Role 3: Junior Machine Learning Engineer

- **Required skills:** Phases 5–7 in more depth: statistics, feature engineering, scikit-learn, PyTorch, experiment tracking (MLflow), model serving, SQL, data pipelines
- **Portfolio:** Projects 4 and 7, both Phase 7 notebooks, plus one paper reproduction or a serious Kaggle write-up
- **Honest note:** plan 3–4 extra months of ML depth beyond this roadmap (for example the full Machine Learning and Deep Learning specializations) and expect stricter requirements, sometimes a master’s degree
- **Titles to search:** Junior ML Engineer, Associate Data Scientist, MLOps Associate

Role 4: AI Full-Stack Developer

- **Required skills:** everything in Role 2 plus confident React and TypeScript and deployment
- **Portfolio:** Projects 8 and 12 as flagship pieces, with live demos
- **Titles to search:** Full-Stack Engineer (AI), Product Engineer, Founding Engineer at early-stage startups

GitHub profile strategy

- Pin the 6 best repositories (your core projects); give the profile README a 3-line summary, your stack and links to live demos
- Every project README follows one template: problem, demo GIF, architecture diagram, stack, how to run, evaluation results, design decisions and trade-offs, limitations, next steps
- Commit steadily (small commits with clear messages); use branches, pull requests and Issues even when working alone; tag releases; show CI badges
- Never commit secrets; add `.env.example`; use topics on every repo
- Make 1–3 small open-source contributions (documentation fixes or issues labelled “good first issue”) in tools you use

Resume preparation

- One page, plain text layout (no tables, columns or images) so applicant tracking systems can read it; export as PDF
- Order: header with links, two-line summary, projects, skills, education, certifications
- Write each project bullet as action + technology + measurable result, using only numbers that are true, for example: “Built a RAG assistant over 20 PDFs with pgvector; retrieval hit rate 84% on a 40-question test set; p95 latency 2.3 s”
- Mirror keywords from each job description honestly; keep one master resume and tailor a copy per role

LinkedIn optimization

- Headline states the role and stack: “AI Application Developer | Python, FastAPI, React, RAG”
- About section in three short paragraphs: what you build, what you have shipped, what you are looking for

- Featured section: live demos and your best repositories
- Post a short build-in-public update every one or two weeks (what you built, one lesson, one number)
- Turn on Open to Work for recruiters; connect with engineers and recruiters at companies that hire juniors, with a short personal note

Interview preparation

- **Coding:** NeetCode 150 Easy and Medium; practice out loud with a timer
- **Python and backend:** async vs threads, the GIL, decorators, REST design, database indexes and transactions, caching, JWT flow, idempotency
- **System design:** design a RAG chatbot, a file-upload and processing pipeline, a rate limiter; use the System Design Primer
- **AI-specific:** RAG failure modes, chunking trade-offs, how you evaluate a RAG system, hallucination mitigation, token cost control, prompt injection, when to fine-tune vs use RAG
- **ML basics:** bias-variance, metrics for imbalanced data, data leakage, regularization
- **Behavioral:** prepare three STAR stories from your projects (a hard bug, a trade-off decision, something that failed and what you changed)
- **Mock interviews:** at least 5 with friends or peers before your first real one

Technical assessment preparation

- **Take-home:** time-box it, ask clarifying questions, write a README with decisions, add tests, make it run with one command
- **Live coding:** think aloud, clarify the problem, start with a simple solution, then optimize
- **Online timed tests:** practice on a coding platform under a timer; follow the company's rules on AI tool use exactly

Freelancing opportunities

- Start after Month 6–7 with small, clearly scoped services: an FAQ chatbot for a business website, document data extraction, internal knowledge search, simple dashboards
- Find work through your network, local businesses, LinkedIn and freelance platforms such as Upwork

- Use written scopes, milestone payments and a simple contract; state limits honestly (models can be wrong) and handle customer data carefully
- Turn every finished job into a short case study; do not underprice yourself into unsustainable work

Remote job preparation

- Remote entry-level roles are very competitive; internships, contract-to-hire and hybrid roles are an easier first step
- Show what remote employers look for: clear written communication, strong READMEs and docs, deployed work, consistent GitHub history, comfort with time-zone overlap
- Know the differences between being an employee and a contractor (taxes, payment method, benefits)
- Look at remote-focused boards such as We Work Remotely, Remote OK and Wellfound, and use LinkedIn's remote filter

15. Final Deliverables, Part 1

Deliverable 1: Complete technology roadmap in learning order

1. Python basics, then Git and GitHub
2. Virtual environments, packaging, project structure, pytest basics
3. OOP, exceptions, files
4. Advanced Python: typing, generators, decorators, async
5. Data structures, algorithms, Big-O (continues in the background)
6. HTTP, REST, JSON
7. SQL and PostgreSQL
8. FastAPI and Pydantic
9. SQLAlchemy and Alembic
10. Authentication: JWT and OAuth
11. API testing, logging, error handling
12. Redis, background jobs, SSE and WebSockets
13. TypeScript, React data fetching, auth in the frontend
14. LLM APIs, prompting, structured output, tool calling

15. Streaming AI chat interface
16. Math essentials, NumPy, Pandas, EDA
17. scikit-learn and model evaluation
18. PyTorch and neural networks
19. Transformers and Hugging Face
20. Embeddings, RAG, pgvector
21. Evaluation and guardrails
22. Agents with LangGraph
23. Docker and Docker Compose
24. GitHub Actions CI/CD
25. Deployment, secrets, monitoring, backups
26. API security, rate limiting, performance
27. AWS fundamentals
28. Capstone project
29. Career materials and interview preparation

Deliverable 2: 12-month learning timeline

See Section 13 above (month-by-month topics, weekly milestones, daily activities, coding practice, project milestones and outcomes).

Deliverable 3: Technology stack recommendation

- **Language and tooling:** Python 3.13 or newer, uv, Ruff, mypy or pyright, pytest, pre-commit
- **Backend:** FastAPI, Pydantic v2, SQLAlchemy 2.0, Alembic
- **Data:** PostgreSQL with pgvector (one database for relational data and vectors); Redis for cache, rate limits and queues
- **Background work:** ARQ or Celery
- **Frontend:** React, TypeScript, Vite, Tailwind CSS, TanStack Query, Zod
- **AI:** one provider SDK used directly, embeddings API, structured outputs with Pydantic, LangGraph for agents (LangChain only for loaders and integrations), your own evaluation scripts or Ragas, PyTorch and Hugging Face for ML work
- **Infrastructure:** Docker and Compose, GitHub Actions, Vercel for the frontend, Render or Railway for the backend, S3-compatible object storage

- **Observability:** Sentry for errors, Langfuse or your own tables for LLM traces and costs
- **Why this stack:** it is typed end to end, widely used in AI startups, cheap to run, and every piece has strong official documentation
- **Reasonable alternatives:** Django (if the employer uses it), Next.js (SEO-heavy sites), Pinecone (managed vector search at larger scale)

16. Final Deliverables, Part 2: Checklists

Deliverable 4: Project portfolio checklist

- ☐ At least 6 of the 8 core projects (★) finished; at least 3 deployed with live URLs
- ☐ Every README has: problem, demo GIF or video, architecture diagram, stack, one-command run instructions, results, design decisions, limitations
- ☐ Tests and a CI badge on at least 3 projects
- ☐ At least one project with real evaluation numbers (RAG hit rate, model metrics)
- ☐ At least one project with auth, a background worker and Docker Compose
- ☐ One ML project with a model card; one agent project with a trace or graph diagram
- ☐ No secrets in the history; `.env.example` present; licence file added
- ☐ A 2-minute demo video for the capstone
- ☐ One written case study (blog post or README section): what went wrong, what you changed, what you measured

Deliverable 5: Essential mathematics checklist

Linear algebra

- ☐ Vector addition, scaling and dot product, and what the dot product means geometrically
- ☐ Cosine similarity, and why it is used for embeddings
- ☐ Matrix shapes, multiplication and transpose; broadcasting in NumPy
- ☐ [SHOULD] Eigenvector intuition and PCA

Calculus

- ☐ Derivative as slope; partial derivatives and the gradient
- ☐ Chain rule intuition (why backpropagation works)

- ☐ Gradient descent and the effect of the learning rate

Probability and statistics

- ☐ Mean, median, variance, standard deviation; normal distribution
- ☐ Conditional probability, independence, Bayes' rule, expected value
- ☐ Sampling, bias, correlation vs causation, basic confidence intervals
- ☐ Precision, recall, F1, ROC-AUC: what each tells you and when each misleads
- ☐ Softmax and cross-entropy intuition

Deliverable 6: GitHub skills checklist

- ☐ init, clone, add, commit, push, pull; readable commit messages
- ☐ Branches, pull requests, code review comments, merge and resolving conflicts
- ☐ Merge vs rebase (concept), stash, revert, safe use of reset
- ☐ `.gitignore`, never committing secrets, what to do if you did (rotate the key)
- ☐ Tags and releases; Issues and a Projects board for your own roadmap
- ☐ A GitHub Actions workflow that runs lint and tests
- ☐ Branch protection, Dependabot alerts and secret scanning turned on
- ☐ Profile README, six pinned repositories, a consistent README template

Deliverable 7: Job-ready skills checklist

Python and CS

- ☐ Idiomatic, typed Python; async basics; solid use of the standard library
- ☐ Easy problems in 15–20 min, Medium problems in 30–45 min; Big-O stated for each

Backend and data

- ☐ Designs REST endpoints with correct status codes and validation
- ☐ Writes SQL joins, indexes and transactions; uses migrations
- ☐ Implements JWT auth, ownership checks and rate limiting
- ☐ Writes pytest tests including failure cases

Frontend

- ☐ Typed React app with data fetching, protected routes, streaming UI and uploads

AI

- ☐ Calls an LLM API with streaming, structured output and tool calling
- ☐ Builds RAG with citations and measures retrieval quality
- ☐ Explains and mitigates hallucination, prompt injection and cost growth
- ☐ Trains and evaluates a classic ML model; fine-tunes a small transformer once

Engineering practice

- ☐ Dockerizes a multi-service app; CI runs on every push
- ☐ Deploys to the cloud with secrets handled correctly, monitoring and backups
- ☐ Writes clear READMEs and design notes; estimates tasks realistically

17. Deliverable 8: Common Mistakes to Avoid

1. **Tutorial hell:** following video after video without building. Fix: the blank-file rule and a project at the end of every phase.
2. **Learning everything at once:** starting LangChain before you can write a clean Python function. Fix: one phase at a time.
3. **Skipping SQL, testing and Git:** they are in every job description and every interview.
4. **Skipping data structures completely:** many companies still screen with them. Fix: 30 minutes, four days a week.
5. **Letting an AI assistant write everything:** you will not build the debugging skill interviews check. Fix: write the first version yourself, then ask for review.
6. **Never deploying:** a localhost project is invisible. Deploy by Month 5 and keep deploying.
7. **No evaluation of LLM apps:** “it looks good” is not engineering. Fix: a test set and a number you track.
8. **Calling LLMs from the browser or committing API keys:** keys leak and bills explode. Keep keys on the server only.
9. **Starting deep learning with heavy math:** learn the intuition first and go deeper only when a project needs it.
10. **Twenty half-finished projects:** finish eight, polish them and document them.
11. **Ignoring cost and latency:** record tokens, cost and response time from your first chatbot onward.
12. **Not reading error messages and docs:** read the traceback fully, then the official docs, then search.
13. **Applying too late:** start in Month 10; interviews are a skill that needs practice too.

14. **Chasing every new framework:** use the checklist in this document; add tools only when a project needs them.
15. **Burning out:** protect sleep, take one rest day a week and keep exercise; consistent 3 hours beats three weeks of 8 hours followed by a break.
16. **Resume as a technology list:** employers hire for evidence; show projects and results.

18. Deliverable 9: Weekly Progress-Tracking Template

Copy this into a notes page each Sunday.

Week number and dates: _____ Current phase and month: _____

Time

- Planned hours: ___ · Actual hours: ___
- Split (target 70/20/10): coding ___% · theory ___% · revision ___%

Learning

- Topics completed this week: _____
- Exercises or problems solved: ___ (DSA: ___)
- Concepts I can explain without notes: _____
- Concepts still unclear: _____

Building

- Project and milestone worked on: _____
- Commits this week: ___ · Tests added: ___ · Deployed? ☐ yes ☐ no
- What I built or shipped: _____
- Demo-able result (link or GIF): _____

Review

- Biggest bug and how I fixed it: _____
- One thing I would do differently: _____
- Revision done (flashcards, rebuilt from memory): ☐ yes ☐ partly ☐ no

Next week

- Three specific goals: 1) _____ 2) _____ 3) _____
- Risks or blockers: _____
- Energy level (1–5): ___ · Rest day taken? ☐ yes ☐ no

Monthly check (end of every month): did I hit the month's project milestone? If not, cut scope before moving on, never skip the exit check.

19. Deliverable 10: Final Assessment, Am I Job-Ready?

Take this at the end of Month 12 (and a lighter version at Months 6 and 9). Do each part without notes, then score yourself honestly or ask a peer to grade it.

Part A: 48-hour build challenge (40 points). Write a new small full-stack AI app from a blank repository: login, upload a document, ask questions with citations, persistent history, tests, Docker Compose, deployed URL, README. Score: works end to end (15), code quality and structure (10), tests and CI (5), deployment (5), README and decisions (5).

Part B: Concept interview (20 points). Answer 10 questions aloud in 2 minutes each. Sample questions:

1. What happens between a user pressing Send and the first token appearing?
2. How do you stop one user from reading another user's documents in a RAG system?
3. Why can chunking ruin retrieval? How would you test it?
4. What is the difference between fine-tuning and RAG, and when do you choose each?
5. How do you reduce hallucinations, and how do you measure that you did?
6. What does a database index do, and what is its cost?
7. Explain the JWT access and refresh token flow and where tokens should be stored.
8. When does `async` help and when does it not?
9. What is data leakage? Give an example.
10. What would you monitor on a production LLM application?

Part C: Coding test (15 points). Three problems in 90 minutes (two Easy, one Medium), solved with correct complexity.

Part D: System design (15 points). Design an AI document assistant for 1,000 users on a whiteboard in 45 minutes: components, data flow, database tables, failure handling, cost and security.

Part E: Portfolio review (10 points). Check against the checklists in Section 16. One point per fully met group of items.

Scoring and what to do next

- **80–100:** you are job-ready for junior AI Application Developer and Backend roles; apply aggressively and keep practicing interviews.
- **60–79:** apply while fixing the weakest part for 3–4 weeks, then retake that part.
- **Below 60:** pause new topics; spend 4–6 weeks on the failing part (usually the build, coding or system design) and retake the whole assessment.

Start today: install Python 3.13 or newer and uv, create your GitHub profile README, and finish the first five Exercism exercises. Then follow Month 1 exactly as written.